

Функциональное программирование в Python



**Senior Software Engineer,
RiotGames**

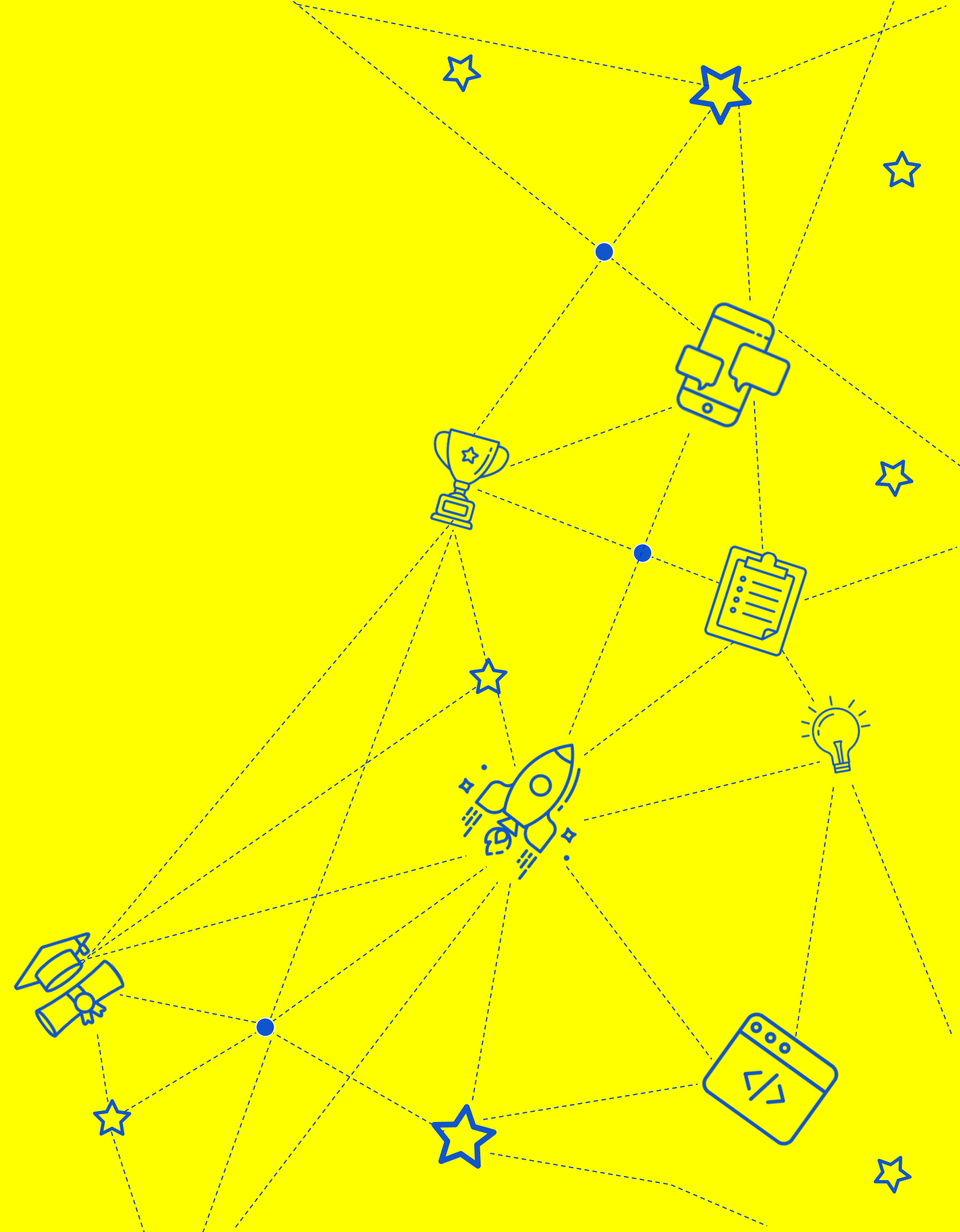
Дмитрий Абрамов

- Построил RTB платформу с аудиторией ~100 млн человек
- Участвовал в создании ряда успешных продуктов за рубежом, от Австралии до США
- Более 10 лет управляю командами разработки в высоконагруженных сложных проектах

Содержание урока

- ★ Функциональная парадигма программирования
- ★ Функции высшего порядка
- ★ Чистые функции
- ★ Lambda функции
- ★ Рекурсия
- ★ Каррирование, замыкание и частичное применение

Функциональная парадигма программирования



Свойства ФП

1

Функции первого класса
и высшего порядка

2

Неизменяемое состояние
и переменные

3

Все вычисления через рекурсию

4

Функциональная прозрачность

5

Все функции являются чистыми

6

Лямбда-исчисление

7

Акцент на том что вычисляем, а не как

Преимущества функционального подхода

В большинстве случаев код становится чище и короче, повышается надежность благодаря преобладанию чистых функций.



Компилятор также может оптимизировать программу, понимая, какие функции вызывать и в каком порядке.



Функциональный код проще тестировать.



Возможности распараллеливания вычислений.



Возможные недостатки

Иногда требуется
большой объем памяти
для вычислений.



Нельзя сделать что-то полезное,
используя только лишь
чистые функции.



Функциональность в Python

Существует великое множество языков, реализующих элементы функциональной парадигмы программирования: от чисто функциональных (Haskell) и до множества императивных.

Несмотря на преобладание императивного подхода функциональная парадигма становится все более популярной.

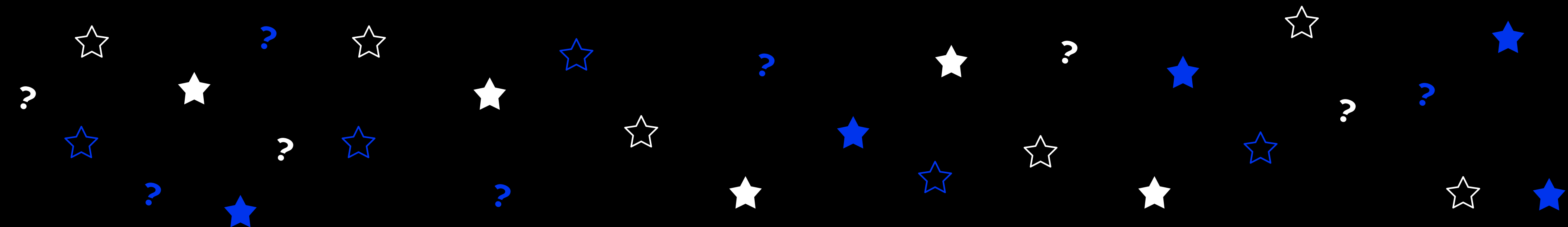
Что насчет функциональности в Python?

Python не является чистым функциональным языком, но позволяет разрабатывать в функциональном стиле, забирая всё лучшее из обеих парадигм.

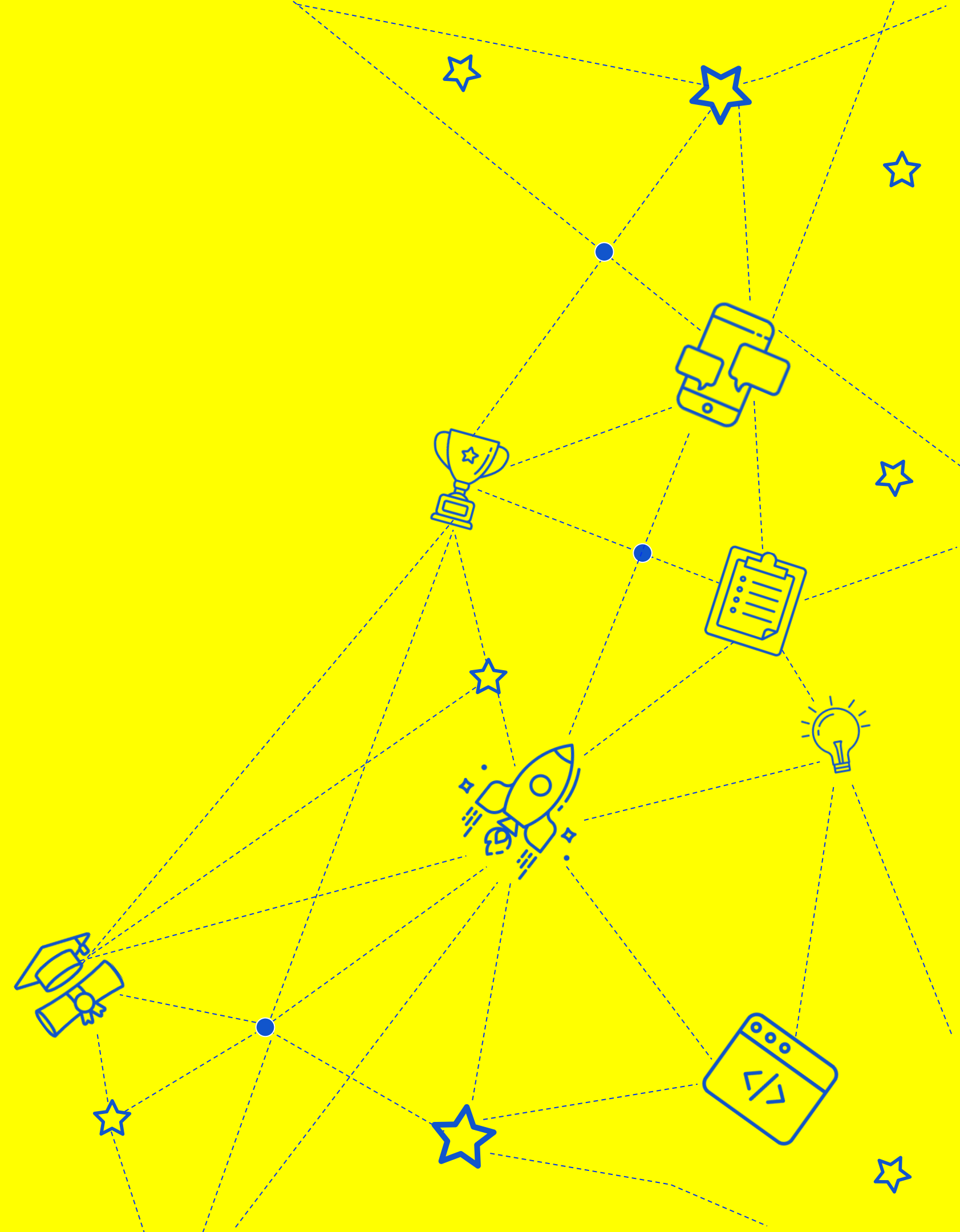


Спасибо за внимание

Продолжение в следующем видео

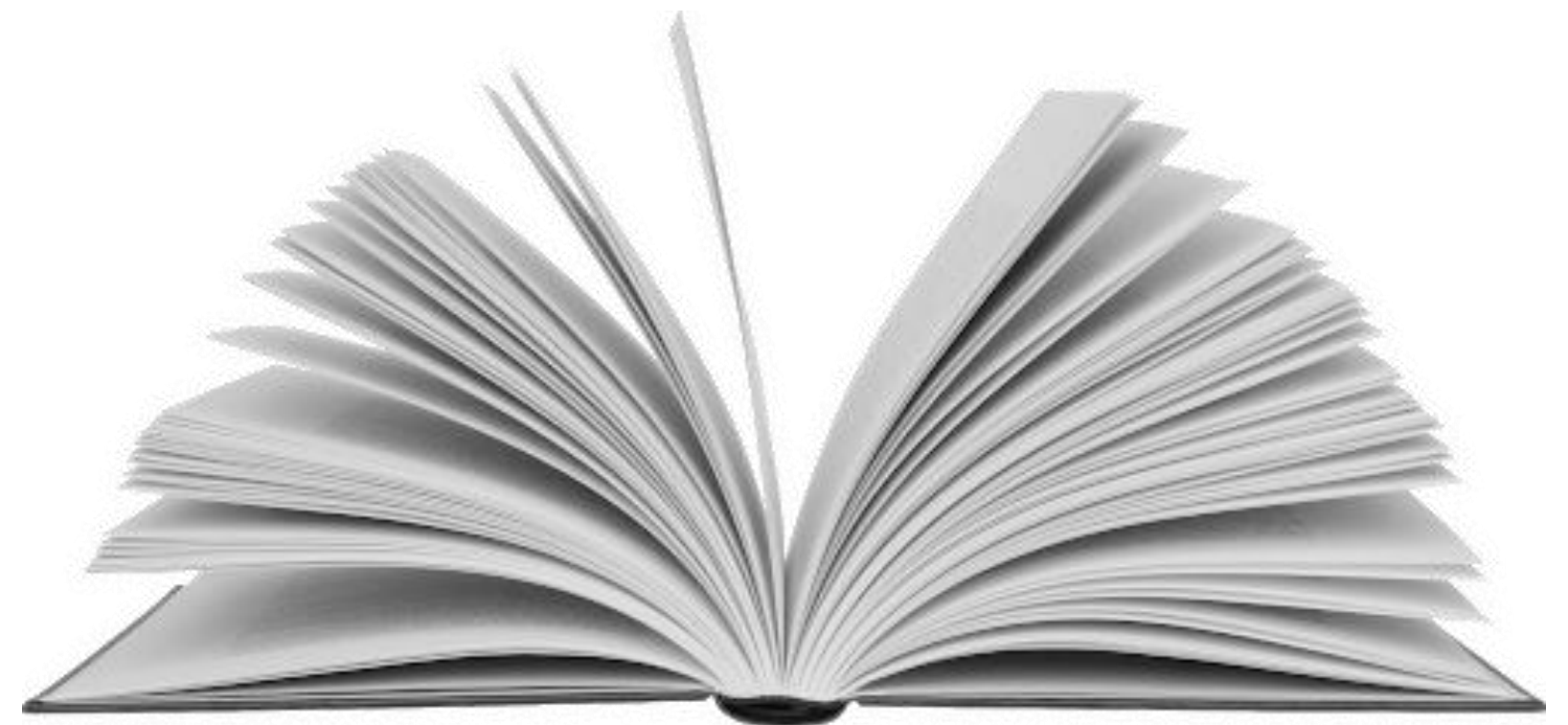


Функции высшего порядка



Определение функции высшего порядка

Функции высшего порядка принимают в качестве аргументов другие функции и/или возвращают другую функцию в качестве результата своей работы.



Пример функции высшего порядка

```
def func1(mul):  
    def func2(value):  
        return mul*value  
    return func2
```

```
temp_fun = func1(3)  
result = temp_fun(2)  
print(result)  
>> 6
```

В примере на первом шаге мы передаем в качестве параметра значение мультипликатора и только на втором передаем значение, которое будет умножено на мультипликатор.

Использование на практике

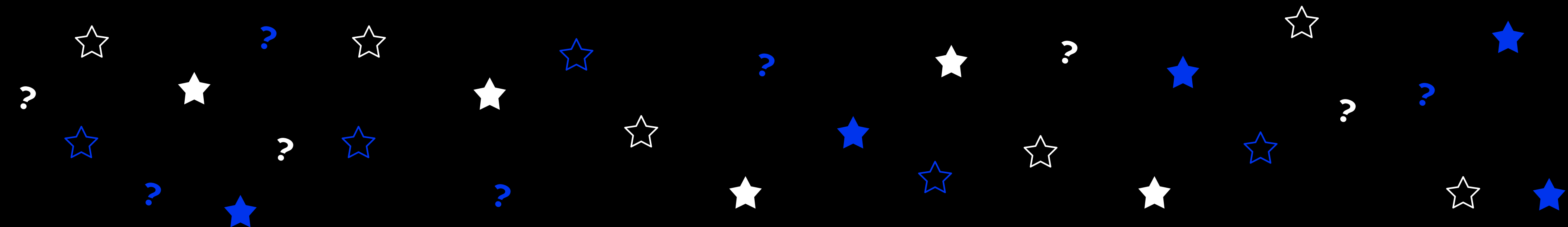


Функции высшего порядка представляют собой мощный инструмент для создания повторно используемого и модульного кода, позволяя функциям принимать другие функции в качестве аргументов или возвращать функции в качестве результатов.

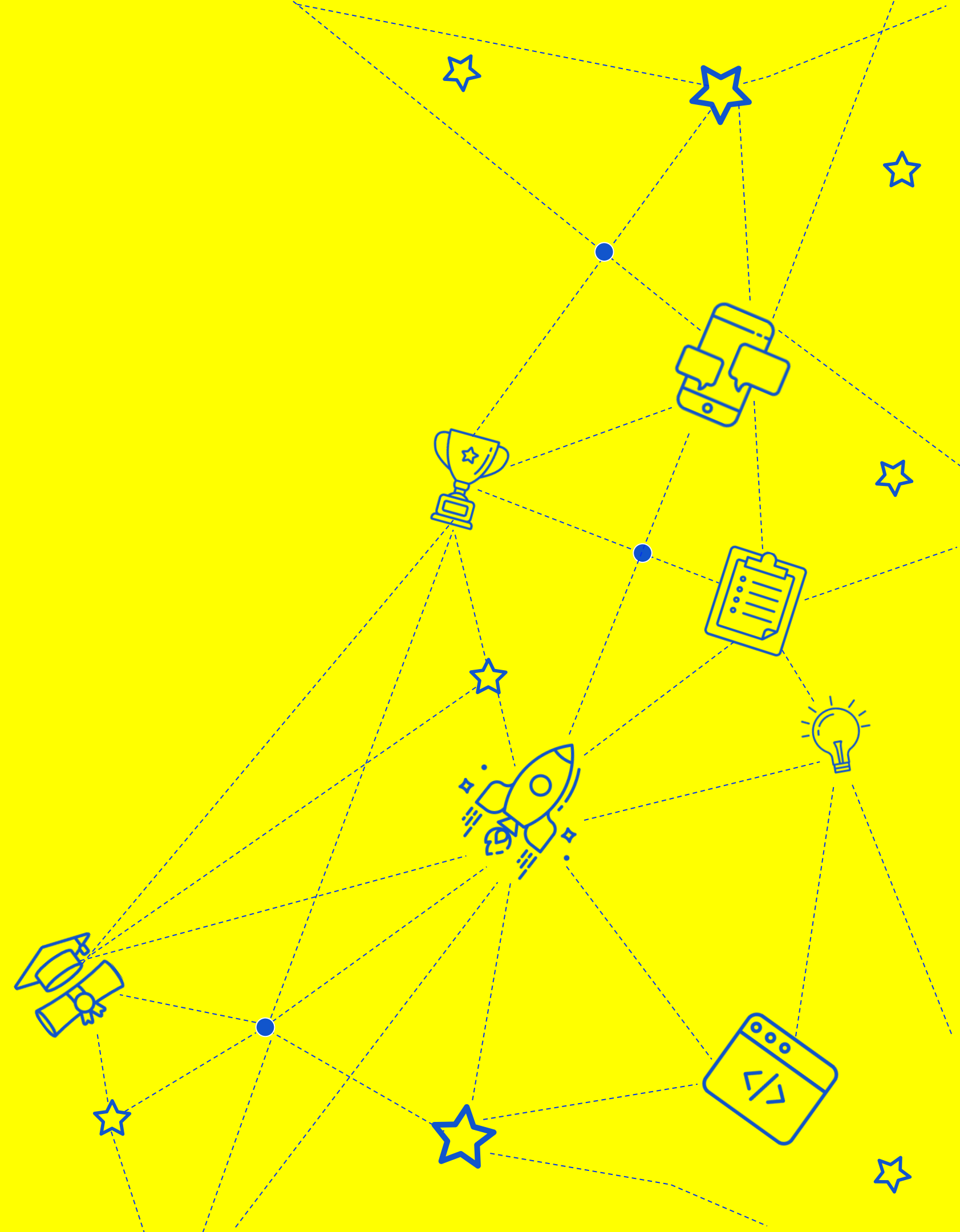
Это может привести к более лаконичному, гибкому и выразительному коду.

Спасибо за внимание

Продолжение в следующем видео



Чистые функции



Понятие чистоты

• Чистая функция

– функция, которая не обладает побочными эффектами.

• Детерминированная (чистая функция)

– функция, которая при идентичном наборе аргументов возвращает один и тот же результат.

Примеры чистой функции и функции с побочными эффектами

Чистая функция

```
def mysum(a, b):  
    return a+b
```

```
print(mysum(1,2))
```

```
>> 3
```

```
print(mysum(1,2))
```

```
>> 3
```

Функция с побочными эффектами

```
from datetime import date
```

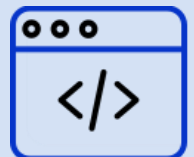
```
def mysum_weekday(a, b):  
    weekday = date.today().weekday()  
    return weekday+a+b
```

```
print(mysum_weekday(1,2))
```

```
>> ?
```

Зачем мне это и когда использовать на практике?

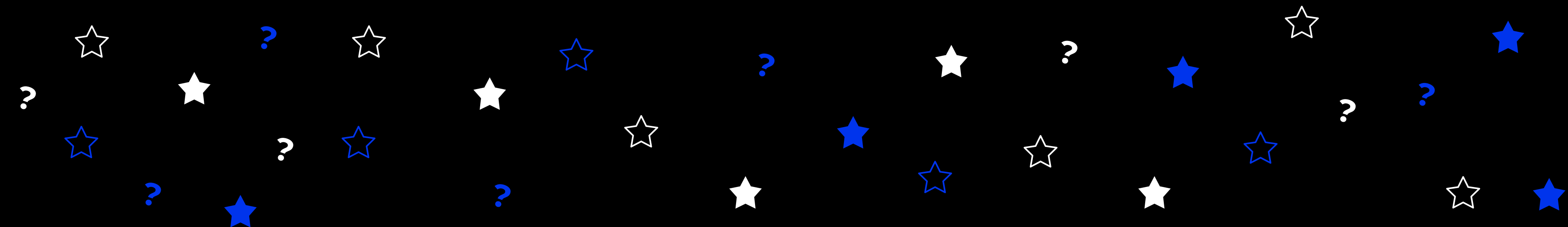
Старайтесь делать большинство своих функций чистыми, тогда вам будет легко их тестировать!



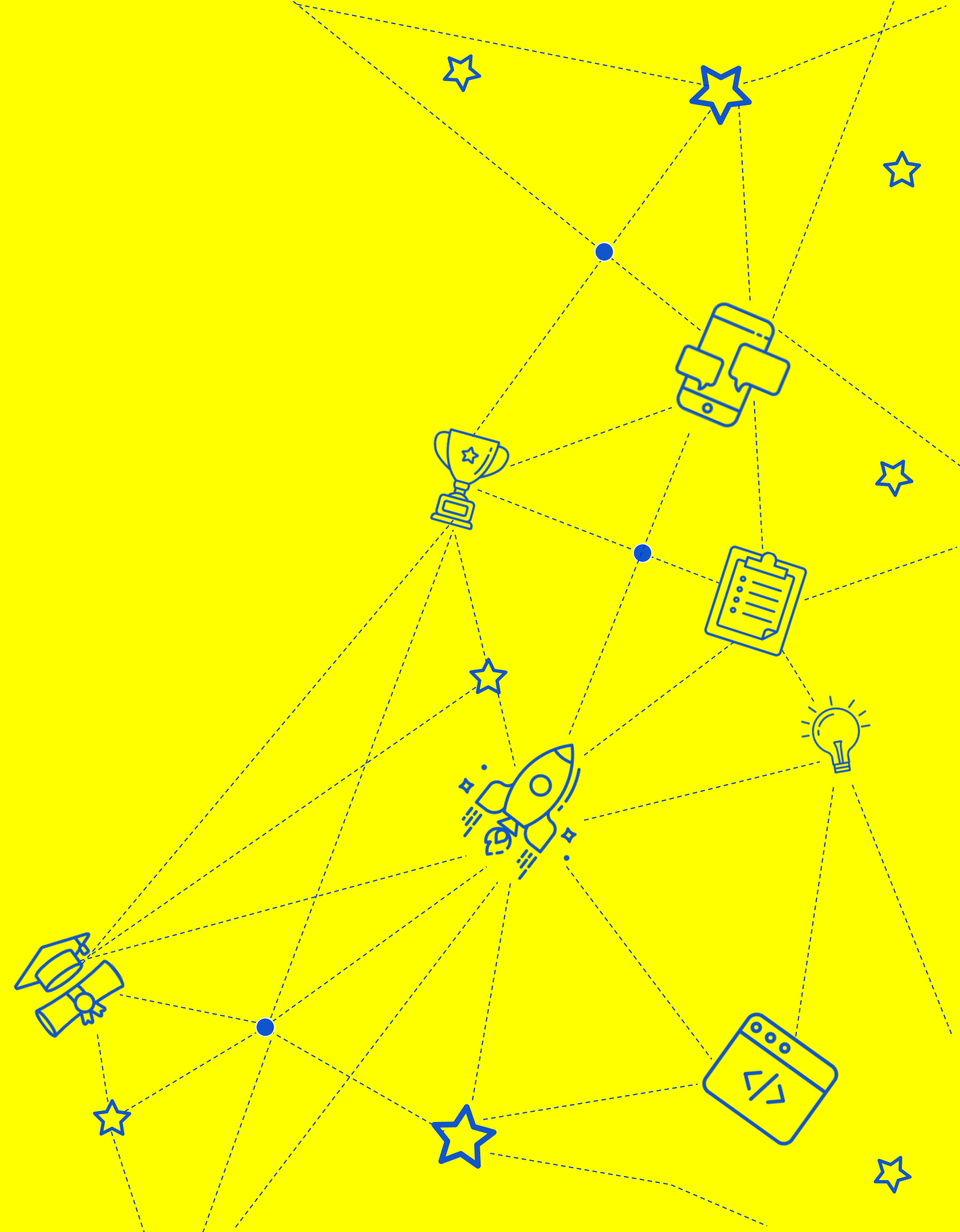
Вы будете благодарны себе за то, что с самого начала стали делать это, ведь куда больший процент времени тратится не на сам код, а на написание новых и исправление существующих тестов при внесении изменений в работающий продукт.

Спасибо за внимание

Продолжение в следующем видео



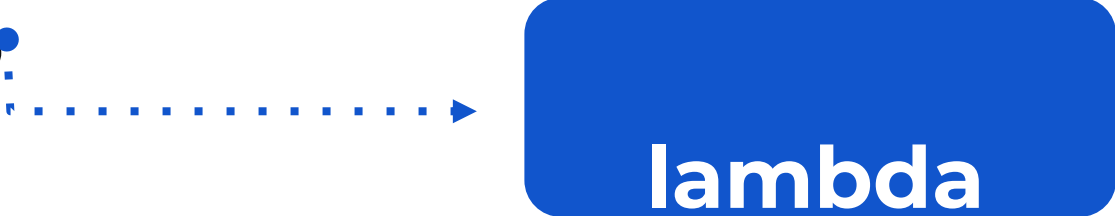
Lambda функции



Определение `lambda` функции

- **Lambda функция** – функция без имени или анонимная функция.

Определяется через ключевое слово:



lambda

Имеет синтаксис:

lambda arguments: expression,

где количество аргументов может быть любым,
но выражение должно быть единственным.

Примеры lambda функций

```
double = lambda x: x*2  
print(double(2))  
>> 4
```

Удваиваем значение

В качестве аргумента в ФВП
(наиболее частый кейс применения)

```
mylist = [1, 4, 5, 78, 33, 10, 54, 17]  
filtered_list = list(filter(lambda x: (x%2 == 0), mylist))  
print(filtered_list)  
>> [4, 78, 10, 54]
```

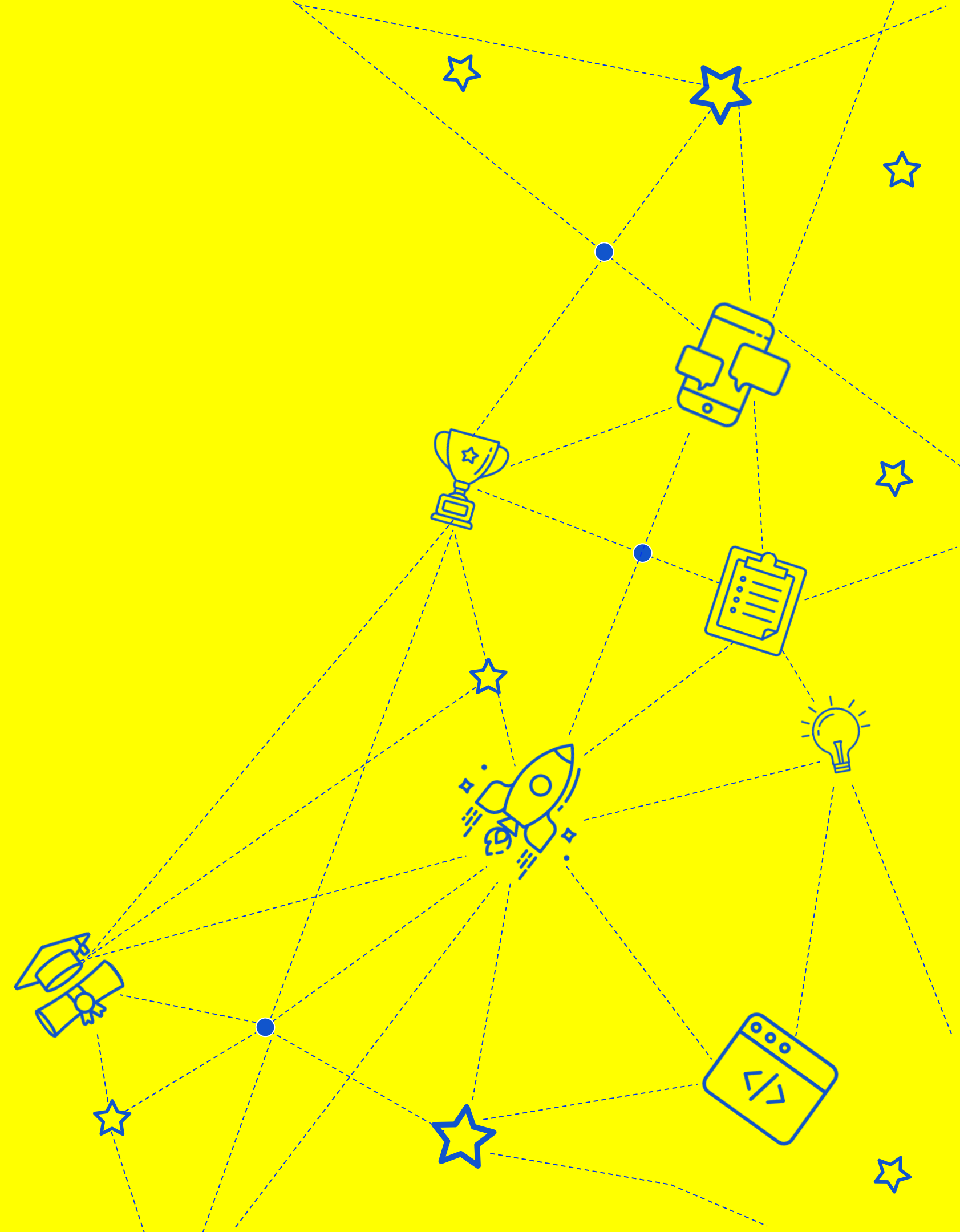
Применение lambda на практике

• Lambda функция

– чрезвычайно полезная конструкция, но старайтесь использовать ее только с простым набором аргументов и простой логикой.



Рекурсия



Сила и красота рекурсии

• Рекурсия

– это вызов функции из нее самой, но с уже измененными параметрами. И это очень просто!



К плюсам рекурсии можно отнести:

- ☐ Рекурсия может сократить время выполнения
- ☐ Ее легко отлаживать
- ☐ Меньший объем кода

Пример рекурсивного подхода

Реализация вычисления факториала без рекурсии

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        product = 1  
        for i in range(1, n+1):  
            product *= i  
        return product  
  
print(factorial(5))  
>> 120
```

Реализация вычисления факториала через рекурсию

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)  
  
print(factorial(5))  
>> 120
```

Использование на практике

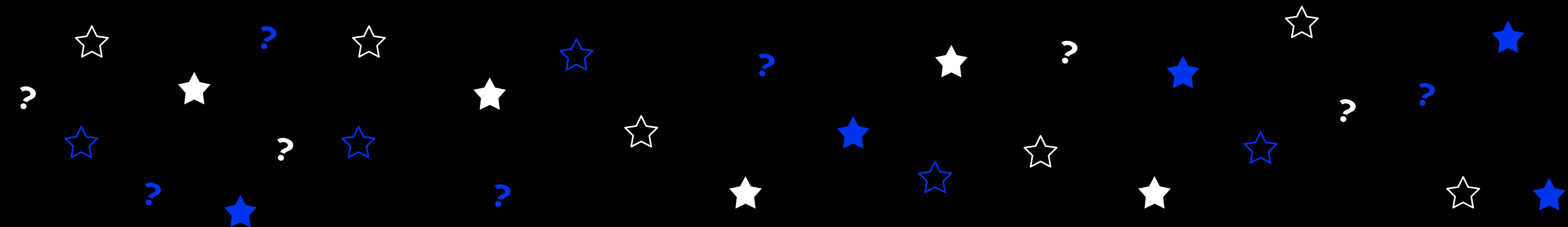


Несмотря на наличие плюсов, в рекурсии есть и минусы:

- ❑ Она может занимать больше места в памяти при условии интенсивных вычислений или работы с данными.
- ❑ Нужно быть аккуратными и следить за стеком.

Спасибо за внимание

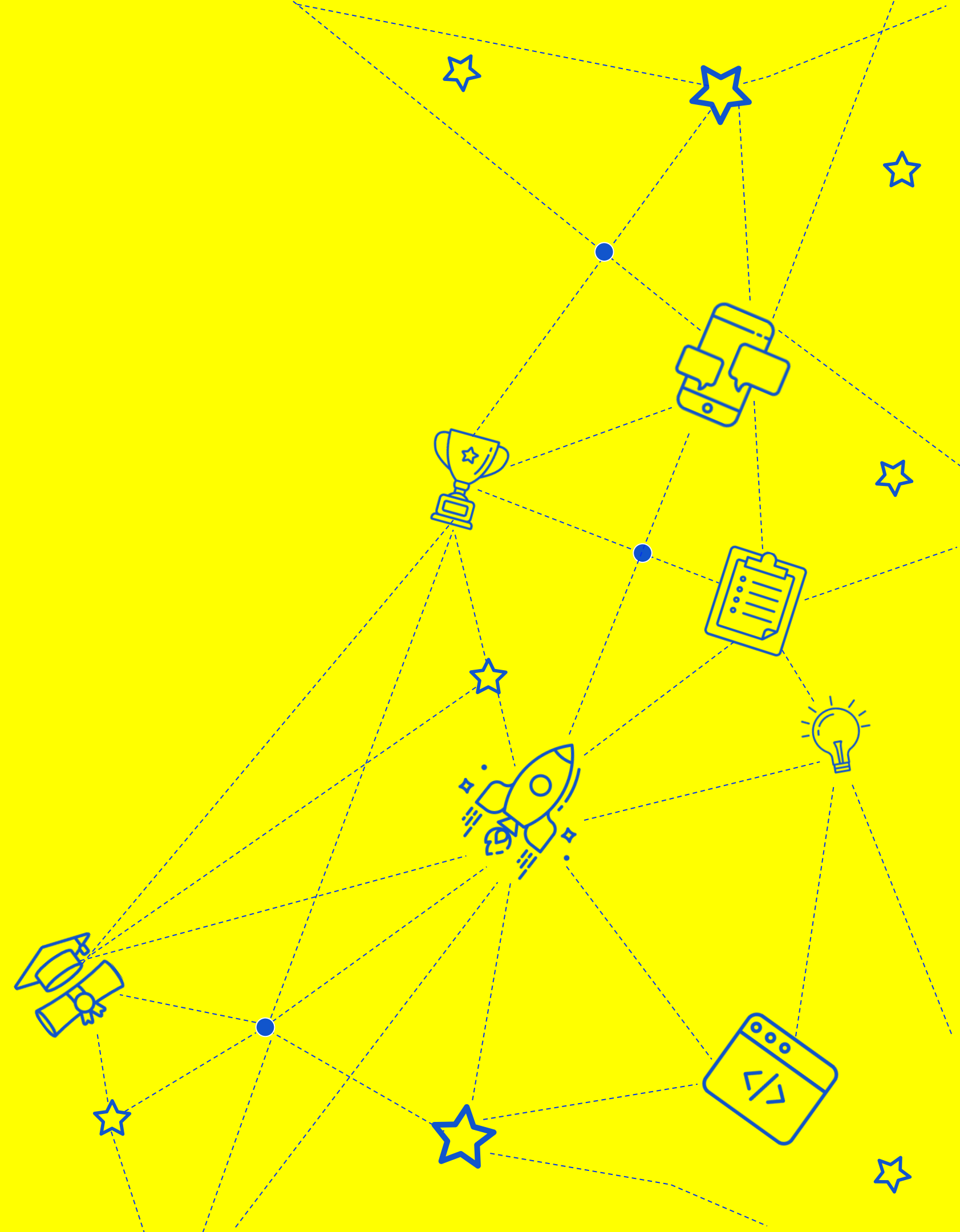
Продолжение в следующем видео



Каррирование, замыкание и частичное применение



О спикере



Каррирование

• Каррирование

— это преобразование функции от множества аргументов в набор функций, каждая из которых является функцией от одного аргумента.

```
def curr_func(a):  
    def tmp_func1(b):  
        def tmp_func2(c):  
            return a+b+c  
        return tmp_func2  
    return tmp_func1
```

```
print(curr_func(1)(2)(3))
```

```
>> 6
```

Частичное применение

Функция, которая принимает все свои аргументы по одному и после каждого аргумента возвращает функцию ожидающую следующий.

```
def curr_func(a):  
    def tmp_func1(b):  
        def tmp_func2(c):  
            return a+b+c  
        return tmp_func2  
    return tmp_func1  
  
partial_result = curr_func(1)(2)  
print(partial_result(3))  
>> 6
```


Замыкание

Функция, в теле которой присутствуют ссылки на переменные, объявленные вне тела этой функции.

```
a = 4
```

```
def func1():  
    return a+1
```

```
def func2(a):  
    b = 1  
    return a+b
```

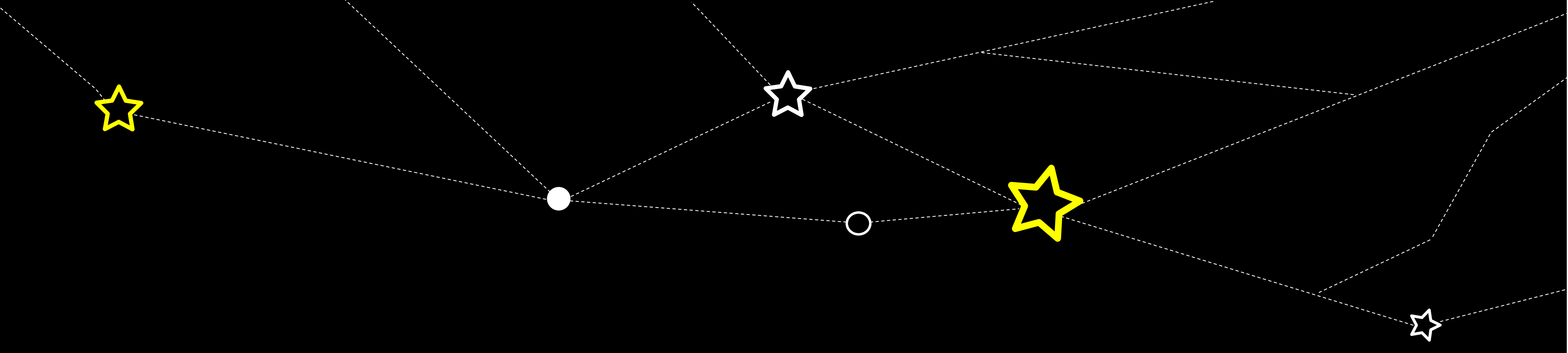
```
def func3():  
    x = 1  
    return lambda: a + x
```

Как я смогу это использовать?

Большинство программ могут быть написаны и без использования функционального подхода, но чем большим набором инструментов и техник вы владеете, тем ценнее вы как специалист!



Попробуйте начать применять эти подходы в своей практике, это может оказаться для вас увлекательным!



СПАСИБО ЗА ВНИМАНИЕ



Dimitry Abramov

Домашнее задание

- ❑ Реализуйте рекурсивную функцию, которая выводит только нечетные числа из переданного списка на консоль.
- ❑ Реализуйте рекурсивную функцию для подсчета элементов в списке.
- ❑ Напишите функцию, которая при каждом следующем своем вызове выводит на консоль следующий элемент заданного списка.

